untangle::actuator

Generated by Doxygen 1.18.0

Chapter 1

actuator

An actuator is a generic callable that may invoke a list of actions(callables of type std::function<...>).

Its goal is to provide a light and simple mechanism to connect components that want to communicate with each other. It works similar as the "signal and slots" mechanisms from other frameworks, but much simplified.

Modern programming languages do not offer class intrinsic interfaces that would improve significantly how objects are connected and how they communicate. Instead, languages, like C++, are using external interfaces, that are not only cluttering the code, but they don't feel "natural" as they are opposed to what an interface is in reality. Basically the objects and interfaces are not separated, and the interface is intrinsic to the object(think on the cogs in a mechanism). A generic callable is an attempt to mitigate this problem. async implementation is such use case.

An actuator provides the results of all actions in a vector, in the same order as the actions were added.

Example: how to use actuator instead of polymorphism

```
void rotate_shapes(const std::vector<shape*>& shapes, int angle)
{
  for (const auto& s : shapes)
  {
    s->rotate(angle);
  }
}

std::shared_ptr<triangle> t(new triangle);
std::shared_ptr<circle> c(new circle);
std::shared_ptr<square> s(new square);

// using polymorphism
std::vector<shape*> shapes;
shapes.push_back(t.get());
shapes.push_back(c.get());
shapes.push_back(s.get());

rotate_shapes(shapes, 10);

// using actuator
auto action1 = untangle::bind(t, &triangle::rotate);
auto action2 = untangle::bind(c, &circle::rotate);
auto action3 = untangle::bind(s, &square::rotate);
auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate(20);
```

For convenience there are provided helpers methods to "connect" to an initial list of "actions", or to create bindings to class methods.

Please check the manual in doc/refman.pdf for further references.

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

Chapter 6

Topic Documentation

6.1 namespace untangle: functions

Functions

- template<typename action_t, typename... Actions>
  actuator< action_t > untangle::connect (action_t &A1, Actions &... An)

  Creates an actuator holding an initial list of actions.
- template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↩
  T>::type>>
  action_t untangle::bind (const std::shared_ptr< class_t > &obj, T class_t::∗method)

  Binding to a class function member.
- template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↩
  T>::type>>
  action_t untangle::bind (class_t ∗obj, T class_t::∗method)

  Binding to a class method.

6.1.1 Detailed Description

6.1.2 Function Documentation

6.1.2.1 connect()

template<typename action_t, typename... Actions>
actuator< action_t > untangle::connect (
    action_t & A1,
    Actions &... An)

Creates an actuator holding an initial list of actions.

Parameters

A1	- The first action. It is specified as std::function<...>.

An	- Any number of further actions, of the same type as A1.

Returns

An actuator.

Example:

```
void rotate_shapes(const std::vector<shape*>& shapes, int angle) {
  for (const auto& s : shapes) {
    s->rotate(angle);
  }
}
  auto action1 = untangle::bind(t, &triangle_mock::rotate);
  auto action2 = untangle::bind(c, &circle_mock::rotate);
  auto action3 = untangle::bind(s, &square_mock::rotate);

  auto actuator_rotate = untangle::connect(action1, action2, action3);
  actuator_rotate(20);
```

6.1.2.2 bind() [1/2]

```
template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<T>::↩
type>>
action_t untangle::bind (
                    const std::shared_ptr< class_t > & obj,
                    T class_t::* method)
```

Binding to a class function member.

It returns a std::function(lambda) that wraps the function member. It may be used to provide an action for connect() or actuator::add().

Remarks

It requires a shared pointer to the class type. A std::weak_ptr to it is captured internally in a lambda, so the binding can always check whether the shared object is still alive without keeping it alive itself. Therefore, it is safe to use actions provided by this binding inside an actuator, and safe for the action to outlive the caller's shared pointer.

Parameters

obj	- Class object.
method	- Pointer to function member. It is specified as &<class type>::<function member>

Returns

action_t - A std::function that wraps the pointer to function member.

Remarks

If the class object gets invalid, invoking this binding will throw an exception of type invalid_action.

6.1.2.3 bind() [2/2]

```
template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<T>::↩
type>>
action_t untangle::bind (
                class_t * obj,
                T class_t::* method)
```

Binding to a class method.

Attention

It is not safe to use this binding when the pointed-to object may be destroyed. A null pointer is detected and reported as a dead action, but a pointer to an already destroyed object can not be distinguished from a valid one. Prefer the overload taking a std::shared_ptr.

Remarks

It is provided for convenience of use: within a class it is safe to create bindings through this pointer.

Binding a null pointer produces an action that is always dead: invoking it throws invalid_action, and an actuator drops it.

Parameters

obj	- Pointer to class.
method	- Pointer to function member. It is specified as &<class type>::<function member>

Returns

action_t - A std::function that wraps the pointer to function member.

Chapter 7

Namespace Documentation

7.1 untangle Namespace Reference

Interface to untangle::actuator functor.

Classes

- struct invalid_action

  Invalid action exception.
- struct actuator

  An actuator is a functor that can trigger a dynamic list of actions (of type std::function<...>).

Functions

- template<typename action_t, typename... Actions>
  actuator< action_t > connect (action_t &A1, Actions &... An)

  Creates an actuator holding an initial list of actions.
- template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↩ T>::type>>
  action_t bind (const std::shared_ptr< class_t > &obj, T class_t::*method)

  Binding to a class function member.
- template<typename class_t, typename T, typename action_t = std::function<typename function_remove_const<↩ T>::type>>
  action_t bind (class_t *obj, T class_t::*method)

  Binding to a class method.

7.1.1 Detailed Description

Interface to untangle::actuator functor.

Chapter 8

Class Documentation

8.1 untangle::actuator< action_t > Struct Template Reference

An actuator is a functor that can trigger a dynamic list of actions (of type std::function<...>).

#include <actuator.hpp>

Public Types

- using actions_t = std::list<action_t∗>

    Actions container type.
- using results_t = std::vector<typename result_t::type>

    Results container type.

Public Member Functions

- actuator & operator= (const actuator &other)=default

    Assignment operator.
- action_t type () const

    Helper method to access the action type of this object.
- void reset ()

    Remove all actions and any stored results.
- template<typename... Args>
  void operator() (Args &&... args)

    The call operator.
- template<typename... Args>
  void invoke_action (const std::string &name, Args &&... args)

    Invokes one single action associated with a key.
- void add (action_t ∗action)

    Add an action to the actions list.
- void add (const std::string &name, action_t ∗action)

    Add action to the actions map associated with a name.
- void remove (const action_t ∗action)

    Remove an action from the actions list.
- void remove (const std::string &name)

    Remove an action from actions map.
- bool is_connected () const

    Check if this actuator is "connected" with other actions.
- bool has_action (const std::string &name) const

    Check if there is certain named action.

Public Attributes

- actions_t actions
  Actions list.
- actions_map_t actions_map
  Named actions map.
- results_t results
  Actions return values list.

8.1.1 Detailed Description

template<typename action_t>
struct untangle::actuator< action_t >

An actuator is a functor that can trigger a dynamic list of actions (of type std::function<...>).

Remarks

An actuator object can be constructed with an initial list of actions by connect().

Template Parameters

action_t	Action type. It is specified as std::function<...>.

8.1.2 Member Typedef Documentation

8.1.2.1 actions_t

template<typename action_t>
using untangle::actuator< action_t >::actions_t = std::list<action_t*>

Actions container type.

Remarks

The elements stored are of pointer type, that is required to implement the remove() operation. std::function supports only equality operator for nullptr (two std::function(s) can not compare).

8.1.2.2 results_t

template<typename action_t>
using untangle::actuator< action_t >::results_t = std::vector<typename result_t::type>

Results container type.

It holds the return values of the actions that have a non-void return type. Upon the actuator invocation, the returns can be extracted from results.

8.1.3 Member Function Documentation

8.1.3.1 operator=()

template<typename action_t>
actuator & untangle::actuator< action_t >::operator= (
const actuator< action_t > & other) [default]

Assignment operator.

Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
untangle::actuator<decltype(actuator_rotate.type())> actuator_rotate_1 = actuator_rotate;
actuator_rotate_1(20);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

8.1.3.2 type()

template<typename action_t>
action_t untangle::actuator< action_t >::type () const [inline]

Helper method to access the action type of this object.

Note

Intended to be used in expressions by decltype(<actuator instance>.type()

Returns

action_t

8.1.3.3 reset()

template<typename action_t>
void untangle::actuator< action_t >::reset () [inline]

Remove all actions and any stored results.

After this call the actuator is empty: actuator::is_connected() returns false.

8.1.3.4 operator()()

template<typename action_t>
template<typename... Args>
void untangle::actuator< action_t >::operator() (
Args &&... args) [inline]

Parameters

args	- Arguments list must match the action arity.

8.1.3.5 invoke_action()

template<typename action_t>
template<typename... Args>
void untangle::actuator< action_t >::invoke_action (
const std::string & name,
Args &&... args) [inline]

Invokes one single action associated with a key.

Parameters

name	- Key associated with the action.
args	- Arguments list must match the action arity.

8.1.3.6 add() [1/2]

template<typename action_t>
void untangle::actuator< action_t >::add (
action_t * action) [inline]

Add an action to the actions list.

Parameters

action	- Action to be added.

Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).Times(2).WillRepeatedly(testing::Return());
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate.add(&action1);
actuator_rotate(20);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```


8.1.3.7 add() [2/2]

template<typename action_t>
void untangle::actuator< action_t >::add (

Parameters

name	- Name of the action.
action	- Action to be added.

8.1.3.8 remove() [1/2]

template<typename action_t>
void untangle::actuator< action_t >::remove (
const action_t * action) [inline]

Remove an action from the actions list.

An invalid action (empty std::function) is implicitly removed when operator()() is invoked.

Parameters

action	- Action to be removed.

Example:

```
const auto t = std::make_shared<triangle_mock>();
const auto c = std::make_shared<circle_mock>();
const auto s = std::make_shared<square_mock>();

EXPECT_CALL(*t, rotate(testing::_)).Times(0);
EXPECT_CALL(*c, rotate(testing::_)).WillOnce(testing::Return());
EXPECT_CALL(*s, rotate(testing::_)).WillOnce(testing::Return());
auto action1 = untangle::bind(t, &triangle_mock::rotate);
auto action2 = untangle::bind(c, &circle_mock::rotate);
auto action3 = untangle::bind(s, &square_mock::rotate);

auto actuator_rotate = untangle::connect(action1, action2, action3);
actuator_rotate.remove(&action1);
actuator_rotate(50);

testing::Mock::VerifyAndClearExpectations(t.get());
testing::Mock::VerifyAndClearExpectations(c.get());
testing::Mock::VerifyAndClearExpectations(s.get());
```

8.1.3.9 remove() [2/2]

template<typename action_t>
void untangle::actuator< action_t >::remove (
const std::string & name) [inline]

Remove an action from actions map.

Parameters

name	- Name of the action to remove.

8.1.3.10 is_connected()

template<typename action_t>
bool untangle::actuator< action_t >::is_connected () const [inline]

Check if this actuator is "connected" with other actions.

Returns

true - if the actuator::actions list is not empty.

false - if the actuator::actions list is empty.

8.1.3.11 has_action()

template<typename action_t>
bool untangle::actuator< action_t >::has_action (
const std::string & name) const [inline]

Check if there is certain named action.

Parameters

name	- Name associated with the action.

Returns

true - if name can be found in actuator::actions_map

false - if name can not be found in actuator::actions_map

The documentation for this struct was generated from the following file:

- actuator.hpp

8.2 untangle::invalid_action Struct Reference

Invalid action exception.

#include <actuator.hpp>

Public Member Functions

- invalid_action (std::string text)

  Construct a new invalid action object.
- const char * what () const noexcept override

  The message text describing the reason of this exception.

8.2.1 Detailed Description

Invalid action exception.

Remarks

An action may be provided as a binding to a class function member, by using untangle::bind(). When the class object gets invalid, invoking the action will raise an exception to this type.

8.2.2 Constructor & Destructor Documentation

8.2.2.1 invalid_action()

untangle::invalid_action::invalid_action (
std::string text) [inline], [explicit]

Construct a new invalid action object.

Parameters

text	- A message text, describing the reason of this exception.

8.2.3 Member Function Documentation

8.2.3.1 what()

const char ∗ untangle::invalid_action::what () const [inline], [override], [noexcept]

The message text describing the reason of this exception.

Returns

const char∗ - The message text.

The documentation for this struct was generated from the following file:

- actuator.hpp

Chapter 9

File Documentation

9.1 actuator.hpp

```
// Copyright (c) 2018 Nicolae Popescu. MIT License.

#pragma once

#include <exception>
#include <functional>
#include <iostream>
#include <list>
#include <map>
#include <memory>
#include <string>
#include <type_traits>
#include <utility>
#include <vector>

namespace untangle {
// exception
struct invalid_action : std::exception {
  explicit invalid_action(std::string text) : message(std::move(text)) {}

  const char* what() const noexcept override { return message.c_str(); }

 private:
  std::string message;
};

template <typename action_t>
struct actuator final {
  using actions_t = std::list<action_t*>;
  using actions_map_t = std::map<std::string, action_t*>;
  using result_t = std::conditional<std::is_void<typename action_t::result_type>::value, int,
                                    typename action_t::result_type>;
  using results_t = std::vector<typename result_t::type>;

  actions_t actions;
  actions_map_t actions_map;
  results_t results;

  actuator() = default;
  actuator(const actuator&) = default;
  actuator(actuator&&) noexcept = default;
  ~actuator() = default;
  actuator& operator=(const actuator& other) = default;
  actuator& operator=(actuator&&) noexcept = default;

  action_t type() const { return nullptr; }

  void reset() {
    actions.clear();
    actions_map.clear();
    results.clear();
  }

  template <typename... Args>
  void operator()(Args&&... args) {
    results.clear();

```

```
    // Dead bindings are collected here and dropped after the loop.
    // The actuator does not own the actions it points at, so it must never
    // write through action_t* into a std::function belonging to the caller.
    std::vector<action_t*> dead_actions;

    for (const auto& action : actions) {
      // A null pointer or an empty std::function can never be invoked. Calling an
      // empty one throws std::bad_function_call, which is not an invalid_action and
      // would escape this operator, so drop it instead of invoking it.
      if (action == nullptr || !*action) {
        dead_actions.push_back(action);
        continue;
      }

      try {
        if constexpr (std::is_same_v<typename action_t::result_type, void>) {
          (*action)(std::forward<Args>(args)...);
        } else {
          results.push_back((*action)(std::forward<Args>(args)...));
        }
      } catch (const invalid_action& ia) {
        std::cout « ia.what() « std::endl;
        dead_actions.push_back(action);
      }
    }

    for (const auto& dead_action : dead_actions) {
      actions.remove(dead_action);
    }
  }

  template <typename... Args>
  void invoke_action(const std::string& name, Args&&... args) {
    results.clear();
    const auto& it = actions_map.find(name);
    if (it != actions_map.end()) {
      try {
        if constexpr (std::is_same_v<typename action_t::result_type, void>) {
          (*it->second)(std::forward<Args>(args)...);
        } else {
          results.push_back((*it->second)(std::forward<Args>(args)...));
        }
      } catch (const invalid_action& ia) {
        std::cout « ia.what() « std::endl;
        actions_map.erase(name);
      }
    }
  }

  void add(action_t* action) { actions.push_back(action); }

  void add(const std::string& name, action_t* action) { actions_map.emplace(name, action); }

  void remove(const action_t* action) {
    actions.remove_if([&action](const auto& a) { return (action == a); });
  }

  void remove(const std::string& name) { actions_map.erase(name); }

  bool is_connected() const { return !actions.empty() || !actions_map.empty(); }

  bool has_action(const std::string& name) const {
    return actions_map.find(name) != actions_map.end();
  }
};

template <typename action_t, typename... Actions>
actuator<action_t> connect(action_t& A1, Actions&... An) {
  using actuator_t = untangle::actuator<action_t>;
  actuator_t actuator;
  actuator.actions = {&A1, &An...};

  // remove empty actions
  actuator.actions.remove_if([](const auto& action) { return (*action == nullptr); });
  return actuator;
}

template <typename actuator_t>
void remove_empty_actions(actuator_t& actuator) {
  std::vector<typename actuator_t::actions_map_t::const_iterator> removable_iterators;
  for (auto it = actuator.actions_map.begin(); it != actuator.actions_map.end(); ++it) {
    if (it->second == nullptr) {
      removable_iterators.push_back(it);
    }
  }
  for (auto it : removable_iterators) {
    actuator.actions_map.erase(it);
```

```
  }
}

template <typename key_t, typename action_t, typename... Actions>
actuator<action_t> connect(std::pair<key_t, action_t*> A1, Actions... An) {
  using actuator_t = untangle::actuator<action_t>;
  actuator_t actuator;
  actuator.actions_map = {A1, An...};

  // remove empty actions
  remove_empty_actions(actuator);
  return actuator;
}

// generic helpers to remove const qualifier from a function type,
// for instance const member functions
template <typename T>
struct function_remove_const;

template <typename R, typename... Args>
struct function_remove_const<R(Args...)> {
  using type = R(Args...);
};

template <typename R, typename... Args>
struct function_remove_const<R(Args...) const> {
  using type = R(Args...);
};


template <typename class_t, typename T,
          typename action_t = std::function<typename function_remove_const<T>::type»
action_t bind(const std::shared_ptr<class_t>& obj, T class_t::* method) {
  return [wp = std::weak_ptr<class_t>(obj), method](auto&&... args) ->
         typename action_t::result_type {
           // lock() also keeps the object alive for the duration of the call
           const auto obj_ = wp.lock();
           if (!obj_) {
             // inform the actuator about dead binding
             throw invalid_action("bind: invalid object");
           }
           return ((*obj_).*method)(std::forward<decltype(args)>(args)...);
         };
}

template <typename class_t, typename T,
          typename action_t = std::function<typename function_remove_const<T>::type»
action_t bind(class_t* obj, T class_t::* method) {
  return [obj, method](auto&&... args) -> typename action_t::result_type {
    if (!obj) {
      // inform the actuator about dead binding
      throw invalid_action("bind: invalid object");
    }
    return ((obj)->*method)(std::forward<decltype(args)>(args)...);
  };
}

}  // namespace untangle
```

Index